Audited by CRED SHIELDS

CredShields

Smart Contract Audit

March 24, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Vouch between February 23, 2026, and March 17, 2026. A retest was performed on March 20, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Pratik Lagaskar (Auditor), Shreyas Koli (Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Vouch

AICPA SOC

SCS Advocates

OWASP

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

Table of Contents **2**

1. Executive Summary ——————————- **4**

State of Security 5

2. The Methodology ——————————- **6**

2.1 Preparation Phase 6

2.1.1 Scope 6

2.1.2 Documentation 6

2.1.3 Audit Goals 7

2.2 Retesting Phase 7

2.3 Vulnerability classification and severity 7

2.4 CredShields staff 9

3. Findings Summary ——————————- **10**

3.1 Findings Overview 10

3.1.1 Vulnerability Summary 10

5. Bug Reports —————————— **12**

Bug ID #C001 [Fixed] 12

Missing Admin Setters in CapitalPool Cause Uninitialized Reward Snapshots 12

Bug ID #M001 [Fixed] 14

Using wrong slippage in _convertScrapedVpls() 14

Bug ID #M002 [Fixed] 15

Immediate unlock may revert / block instant withdrawals when contract lacks vPLS balance 15

Bug ID #L001 [Acknowledged] 17

Floating and Outdated Pragma 17

Bug ID #G001 [Acknowledged] 18

Cheaper Inequalities in if() 18

Bug ID #G002 [Acknowledged] 21

Cheaper conditional operators 21

Fuzzing Vulnerabilities 25

Bug ID #MED-01 [Fixed] 25

Emission Rate Change Retroactively Applied to Un-Settled Time 25

Bug ID #DO-01 [Fixed] 27

Zero-Value Unlock from Share-to-vPLS Integer Rounding (CapitalPool) 27

Bug ID #DO-02 [Acknowledged] 29

Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (VouchStaking) 29

Bug ID #DO-03 [Acknowledged] 31

Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (CapitalPool) 31

Bug ID #DO-04 [Acknowledged] 33
Unrestricted receive() Absorbs Native PLS into Holder Rewards 33
Bug ID #DO-05 [Acknowledged] 34
exit() Permanently Locks Forfeited Holder Rewards in Vault 34
6. The Disclosure ——————————- 35

1. Executive Summary ----------

Vouch engaged CredShields to perform a smart contract audit from February 23, 2026, to March 17, 2026. During this timeframe, 12 vulnerabilities were identified. **A retest was performed on March 20, 2026, and all the bugs have been addressed.**

During the audit, 1 vulnerability was found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Vouch" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Capital Pool Smart Contract	1	0	2	1	0	2	**6**
Fuzzing Suite	0	0	1	0	5	0	**6**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Capital Pool Smart Contract's scope during the testing window while abiding by the policies set forth by Vouch's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Vouch's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Vouch can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Vouch can future-proof its security posture and protect its assets.

2. The Methodology ----------

Vouch engaged CredShields to perform a Capital Pool Smart Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.
The team also performed extensive fuzz testing to find edge cases and vulnerabilities that a manual code review usually misses.

A testing window from February 23, 2026, to March 17, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Audited Scope: https://github.com/Vouchrun/vouch-staking/tree/a95af622eb74373f4e71b279beb2ba498c449e97 **Retested Scope:** https://github.com/Vouchrun/vouch-staking/tree/809692374ffa49adc2f260735af4dccd758a289f

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Vouch is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 12 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Missing Admin Setters in CapitalPool Cause Uninitialized Reward Snapshots	Critical	Missing functionality (SCWE-006)	Fixed
Using wrong slippage in _convertScrapedVpls()	Medium	Business Logic (SC03-LogicErrors)	Fixed
Immediate unlock may revert / block instant withdrawals when contract lacks vPLS balance	Medium	Business Logic (SC03-LogicErrors)	Fixed
Floating and Outdated Pragma	Low	Floating Pragma (SCWE-060)	Acknowledged
Cheaper Inequalities in if()	Gas	Gas Optimization (SCWE-082)	Acknowledged
Cheaper conditional operators	Gas	Gas Optimization (SCWE-082)	Acknowledged
Emission Rate Change Retroactively Applied to Un-Settled Time	Medium	Business Logic	Fixed
Zero-Value Unlock from Share-to-vPLS Integer Rounding (CapitalPool)	Design Observation	Health Check	Fixed
Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (VouchStaking)	Design Observation	Centralization Risk	Acknowledged

Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (CapitalPool)	Design Observation	Centralization Risk	Acknowledged
Unrestricted receive() Absorbs Native PLS into Holder Rewards	Design Observation	Health Check	Acknowledged
exit() Permanently Locks Forfeited Holder Rewards in Vault	Design Observation	Health Check	Acknowledged

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #C001 [Fixed]

Missing Admin Setters in CapitalPool Cause Uninitialized Reward Snapshots

Vulnerability Type
Missing functionality (SCWE-006)

Severity
Critical

Description
The CapitalPool contract is missing the setVouchStaking() and setStakingRewardPool() admin setters, and the vouchStaking state variable has no constructor initialization either. As a result, once deployed, CapitalPool.vouchStaking permanently remains address(0). The internal helper _claimVouchStakingRewards() (CapitalPool.sol) guards its call with if (address(vouchStaking) != address(0)), which means every deposit path (depositVpls, depositPls) silently skips the claimCapitalFor() callback to VouchStaking. Consequently, when new users deposit into the CapitalPool, their per-share reward snapshots in VouchStaking (userInfo[pid][user].liqLastAccVouchPerShare, liqLastAccVplsPerShare, liqLastAccWplsPerShare) are never initialized and remain at zero. Meanwhile, the pool-level accumulators (pool.liqAccVouchPerShare, etc.) continue growing from the moment addCapitalPool() was called. On the user's first claim() or claimAll() on VouchStaking, the reward calculation reward = (pool.liqAccPerShare - 0) * userShares / 1e12 produces a massively inflated backdated payout spanning the entire lifetime of the pool.

Affected Code
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/CapitalPool.sol

Impacts
The _claimCapital() function in VouchStaking attempts to transfer these phantom amounts via ILPRewardPool(pool.rewardsPool).pullTokenTo(...). Since multiple PIDs share the same StakingRewardPool, a successful phantom claim would drain rewards intended for legitimate stakers across all pools sharing that reward pool.

Remediation

It is suggested to restore the missing admin setters with proper access control, and initialize vouchStaking to the live VouchStaking address (either via the constructor or via a factory-wired immutable/storage slot). The setters must be gated by onlyAdmin.

Retest

This issue is fixed by adding setter functions in CapitalPool.

Bug ID #M001 [Fixed]

Using wrong slippage in _convertScrapedVpls()

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
In _convertScrapedVpls(), when the market path is chosen (useMarket == true), the contract calls compareMarketVsRedeem(_amount), which returns marketValue (expected PLS from DEX) and redeemValue (expected PLS from redemption). The market path is used only when marketValue > redeemValue. Despite that, the DEX swap uses redeemValue as the minimum amount out for swapExactTokensForETH instead of marketValue. The minimum out should reflect the expected output of the chosen route (market), not the alternative route (redeem). Using redeemValue here weakens slippage protection and allows the swap to accept any amount down to the redeem path's value while the protocol intended to benefit from the better market path.

Affected Code
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/CapitalPool.sol#L728-L765

Impacts
The contract can receive significantly less PLS than the quoted market value (e.g. due to sandwiching or normal slippage) and still succeed, as long as the received amount is ≥ redeemValue. Protocol/yield accrual receives less PLS than expected from the chosen route, and the pool is more exposed to MEV and unfavorable execution.

Remediation
To fix this issue when useMarket is true, use marketValue as slippage in swapExactTokensForETH() function.

Retest
Fixed in 8096923.

Bug ID #M002 [Fixed]

Immediate unlock may revert / block instant withdrawals when contract lacks vPLS balance

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Medium

Description
The CapitalPool.startUnlock and CapitalPool.finalizeUnlock functions allow users to burn shares in exchange for vPLS. When unlockPeriod == 0, CapitalPool.startUnlock is designed to immediately transfer the calculated vplsAmount to the caller. When unlockPeriod > 0, the user must later call CapitalPool.finalizeUnlock to receive vPLS after the waiting period. In both flows, if the contract's on-hand vPLS balance is insufficient, _tryMintFromBuffer() is invoked before rechecking the balance.
The issue is that _tryMintFromBuffer() depends on plsMintBuffer, which is only increased via depositPls, and is reset to zero immediately after minting. In typical system states, plsMintBuffer will be zero or below the mint threshold. As a result, when the contract's vPLS balance is lower than vplsAmount, _tryMintFromBuffer() produces no additional vPLS and the subsequent balance check fails, causing a revert with InsufficientVplsBalance(). The root cause is that both immediate and delayed withdrawal paths rely solely on transient contract liquidity and a cleared mint buffer, without a guaranteed liquidity source or fallback mechanism.

Affected Code
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/CapitalPool.sol#L369

Impacts
Users attempting to withdraw—either instantly via startUnlock when unlockPeriod == 0 or later via finalizeUnlock—may have their transactions reverted if the contract lacks sufficient vPLS balance. This results in a denial of withdrawal.

Remediation
Ensure withdrawals do not revert due to temporary vPLS illiquidity by introducing a deterministic liquidity sourcing mechanism or a non-reverting fallback path.

Retest

Fixed in 8096923.

Bug ID #L001 [Acknowledged]

Floating and Outdated Pragma

Vulnerability Type
Floating Pragma (SCWE-060)

Severity
Low

Description
Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., ^0.8.26. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/CapitalPoolFactory.sol#L2
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/CapitalPool.sol#L2

Impacts
If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.
Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.
The likelihood of exploitation is low.

Remediation
Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.29 pragma version
Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/

Retest
Client's comment: Acknowledged.

Bug ID #G001 [Acknowledged]

Cheaper Inequalities in if()

Vulnerability Type
Gas Optimization (SCWE-082)

Severity
Gas

Description
The contract was found to be doing comparisons using inequalities inside the "if" statement. When inside the "if" statements, non-strict inequalities (>=, <=) are usually cheaper than the strict equalities (>, <).

Affected Code

- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L365-L365
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L397-L397
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L532-L532
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L635-L635
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L820-L820
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1019-L1019
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L821-L821
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1020-L1020
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L965-L965
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1044-L1044
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1064-L1064

- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1549-L1549
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1068-L1068
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1553-L1553
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1072-L1072
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1557-L1557
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1076-L1076
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1561-L1561
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1097-L1097
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1578-L1578
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1101-L1101
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1130-L1130
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1159-L1159
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1203-L1203
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1244-L1244
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1447-L1447
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1454-L1454
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1457-L1457
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1460-L1460
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1490-L1490
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1498-L1498
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1506-L1506

- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1584-L1584
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1616-L1616
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1620-L1620
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1624-L1624
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1648-L1648
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1649-L1649
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1650-L1650
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1677-L1677
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1678-L1678
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1679-L1679
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1790-L1790
- https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1807-L1807

Impacts
Using strict inequalities inside "if" statements costs more gas.

Remediation
It is recommended to go through the code logic, and, **if possible**, modify the strict inequalities with the non-strict ones to save gas as long as the logic of the code is not affected.

Retest:
Client's comment: Acknowledged.

Bug ID #G002 [Acknowledged]

Cheaper conditional operators

Vulnerability Type
Gas Optimization (SCWE-082)

Severity
Gas

Description
Upon reviewing the code, it has been observed that the contract uses conditional statements involving comparisons with unsigned integer variables. Specifically, the contract employs the conditional operators x != 0 and x > 0 interchangeably. However, it's important to note that during compilation, x != 0 is generally more cost-effective than x > 0 for unsigned integers within conditional statements.

Affected Code

7. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L397-L397
8. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1064-L1064
9. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1549-L1549
10. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1068-L1068
11. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1553-L1553
12. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1072-L1072
13. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1557-L1557
14. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1076-L1076
15. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1561-L1561
16. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1130-L1130

17. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1132-L1132
18. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1133-L1133
19. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1134-L1134
20. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1159-L1159
21. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1161-L1161
22. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1205-L1205
23. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1246-L1246
24. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1166-L1166
25. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1170-L1170
26. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1174-L1174
27. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1203-L1203
28. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1210-L1210
29. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1211-L1211
30. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1212-L1212
31. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1244-L1244
32. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1251-L1251
33. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1252-L1252
34. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1253-L1253
35. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1447-L1447
36. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1450-L1450
37. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1451-L1451

38. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1452-L1452
39. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1454-L1454
40. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1457-L1457
41. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1460-L1460
42. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1490-L1490
43. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1491-L1491
44. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1498-L1498
45. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1499-L1499
46. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1506-L1506
47. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1508-L1508
48. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1586-L1586
49. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1616-L1616
50. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1620-L1620
51. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1624-L1624
52. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1648-L1648
53. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1649-L1649
54. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1650-L1650
55. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1677-L1677
56. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1678-L1678
57. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1679-L1679
58. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1787-L1787

59. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1790-L1790
60. https://github.com/Vouchrun/vouch-staking/blob/a95af622eb74373f4e71b279beb2ba498c449e97/contracts/VouchStaking.sol#L1792-L1792

Impacts

Employing x != 0 in conditional statements can result in reduced gas consumption compared to using x > 0. This optimization contributes to cost-effectiveness in contract interactions.

Remediation

Whenever possible, use the x != 0 conditional operator instead of x > 0 for unsigned integer variables in conditional statements.

Retest

Client's comment: Acknowledged.

Fuzzing Vulnerabilities

Bug ID #MED-01 [Fixed]

Emission Rate Change Retroactively Applied to Un-Settled Time

Vulnerability Type
Business Logic

Severity
Medium

Description
setRewardPoolEmissions() modifies per-year emission rates without calling _updateAllPoolsForRewardPool() to settle accumulators at the old rate first. The accumulator function _calcAccStandardTriple computes vouchPerSec x timeDiff, where timeDiff spans from pool.stdLastCalcTime to block.timestamp. If any time has elapsed since the last update, the new rate is applied to the entire elapsed interval, including time when the old rate was in effect. The correct pattern already exists in the same contract: set() at L967 calls _updateAllPoolsForRewardPool before any parameter change. A concrete proof-of-concept demonstrates a 9.99x overpayment: a user stakes 100 VOUCH at 1,000 VOUCH/year, waits 7 days (expected ~19.178 VOUCH), then the admin changes the rate to 10,000 VOUCH/year. Without settling accumulators first, the user claims ~191.78 VOUCH. The fuzz test confirms this across 2,000 random parameter combinations.

Affected Code
contracts/VouchStaking.sol - setRewardPoolEmissions()(L1031-1038)

Proof of Concept
Test: test_EmissionRateRetroactive_Concrete

1. Alice stakes 100 VOUCH
2. 7 days pass at rate = 1,000 VOUCH/year
3. Expected rewards: 1,000 × 7/365 ≈ 19.178 VOUCH
4. Admin changes rate to 10,000 VOUCH/year via setRewardPoolEmissions
5. Alice claims → receives 191.78 VOUCH (9.99× overpayment)

Expected at old rate 19.178 VOUCH

Actually claimed	191.78 VOUCH
Overpayment multiplier	9.99×

Impacts

- **Reward miscalculation on rate changes.** On a rate increase, all stakers receive `newRate/oldRate` times more rewards than earned for the elapsed period. On a rate decrease, stakers receive proportionally less than earned.
- **Reward pool depletion.** The `StakingRewardPool` may be drained faster than intended, causing later claimers to face `InsufficientReward` reverts.
- **Routine admin action.** `setRewardPoolEmissions` is an operational call for adjusting protocol emissions — any rate change triggers this behavior.

Remediation

Add _updateAllPoolsForRewardPool(_rewardPool) before modifying the emission rates in setRewardPoolEmissions(), settling all accumulators at the old rates first. This mirrors the existing correct pattern in set().

```
function setRewardPoolEmissions(address _rewardPool, ...) external onlyAdmin {
  RewardPoolConfig storage cfg = rewardPoolConfig[_rewardPool];
  if (!cfg.initialized) revert ConfigNotInitialized();
+  _updateAllPoolsForRewardPool(_rewardPool);  // Settle at old rates first
  cfg.baseVouchPerYear = _vouchPerYear;
  cfg.baseVplsPerYear  = _vplsPerYear;
  cfg.baseWplsPerYear  = _wplsPerYear;
  emit RewardPoolRatesAutoUpdated(_rewardPool, _vouchPerYear, _vplsPerYear,
_wplsPerYear);
}
```

Retest

Patched in vouch-staking commit eb4d740 by settling reward-pool accumulators before applying the new emission rates.

Bug ID #DO-01 [Fixed]

Zero-Value Unlock from Share-to-vPLS Integer Rounding (CapitalPool)

Vulnerability Type
Health Check

Severity
Design Observation

Description
In CapitalPool.startUnlock(), the integer division (_shares * totalVplsDeposited) / totalShares truncates to zero for very small share counts when totalShares is significantly larger than totalVplsDeposited. This state can arise after yield scraping via _accrueYield(). The function proceeds without reverting: shares are deducted, a zero-value unlock request is stored, and finalizeUnlock() later transfers 0 vPLS. The fuzzer confirmed this after a large deposit followed by a 37.29% yield accrual (bumpMintRatio(3729)), where unlocking exactly 1 share produces vplsAmount = 0. This is standard integer arithmetic behavior in Solidity and only manifests when unlocking extremely small share quantities relative to the total share supply.
Observed behaviour:

1. depositVpls(actor, 7.87e62 ether) — Deposit vPLS, receive proportional shares
2. bumpMintRatio(3729 bps) — 37.29% yield → accrueYield reduces TVD, not shares
3. startUnlock(actor, 1 share) — vplsAmount = 1 × TVD / totalShares = 0
   Share deducted, 0 vPLS recorded

Affected Code
contracts/CapitalPool.sol - startUnlock(uint256 _shares)

Impacts
Users who unlock very small share amounts (e.g. 1 share when totalShares is large relative to totalVplsDeposited) lose those shares permanently while receiving 0 vPLS. This is an edge case inherent to integer-division-based share accounting.

Remediation
Add a minimum output guard: if (vplsAmount == 0) revert ZeroAmount() to protect users from accidentally unlocking dust amounts.

Retest

Patched in vouch-staking commit eb4d740 by preventing unlocks that round to vplsAmount == 0 from burning user shares.

Bug ID #DO-02 [Acknowledged]

Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (VouchStaking)

Vulnerability Type
Centralization Risk

Severity
Design Observation

Description
VouchStaking stores only startTime in unlock requests. Both getUnlock() and _finalizeUnlock() compute the deadline dynamically as startTime + standardUnlockPeriod, using the current global value. When the admin adjusts standardUnlockPeriod, all existing unlock requests are affected by the new period, extending or shortening the effective deadline. The fuzzer shrunk a violation to 4 calls: stake, set low unlock period, startUnlock, then raise period. The effective withdrawal deadline shifted by ~5.2 hours in one example. This is an intended admin functionality within the trusted-admin governance model. Mitigating factors: MAX_UNLOCK_PERIOD caps at 14 days (default 5 days), admin is trusted under DAO governance with timelocks, and users can call cancelUnlock() as an escape hatch.

Observed Behaviour:

1. stake(actor, VOUCH_STD, 7330)
2. setStandardUnlockPeriod(221 seconds)
3. startUnlock(actor, VOUCH_STD, full balance)
    a. → unlockAt = T + 221 = 6,941,441
4. setStandardUnlockPeriod(5.09e49 seconds)
    a. → unlockAt now = T + 5.09e49 = 7,104,792
    b. → INVARIANT TRIGGER: 7,104,792 > 6,941,441

Affected Code
contracts/VouchStaking.sol - setStandardUnlockPeriod(), _finalizeUnlock(), getUnlock()

Impacts
Admin adjustments to standardUnlockPeriod retroactively extend or shorten all pending unlock deadlines. This could delay expected withdrawals for users with active unlock requests.

Remediation

Optional enhancement: snapshot the deadline at lock time by storing unlockAt = block.timestamp + standardUnlockPeriod in the unlock request struct and checking against req.unlockAt in _finalizeUnlock().

Retest

Acknowledged as intended behavior under the trusted-admin governance model.

Bug ID #DO-03 [Acknowledged]

Unlock Deadline Dynamically Tied to Global Period - Admin-Controlled (CapitalPool)

Vulnerability Type
Centralization Risk

Severity
Design Observation

Description
Same design pattern as DO-02, in the CapitalPool contract. CapitalPool.finalizeUnlock() checks block.timestamp < req.startTime + unlockPeriod using the current global unlockPeriod. When the admin adjusts unlockPeriod, all pending unlock requests are affected. Capital pools default to 0 days unlock period on deployment and can be set up to 14 days on a per-pool basis. A concrete test demonstrates: with unlockPeriod = 3 days, Alice starts an unlock, then admin changes to 14 days. Alice cannot finalize even after 3 days, being forced to wait an additional 11 days (950,400 seconds). Setting unlockPeriod = 0 also enables instant finalization of all pending unlocks.

Observed behaviour:
Test: test_UnlockPeriodRetroactiveExtension

1. unlockPeriod = 3 days
2. Alice starts unlock → effective deadline at startTime + 3 days
3. Admin sets unlockPeriod = 14 days
4. Time reaches startTime + 3 days + 1 second
5. Alice calls finalizeUnlock() → REVERTS (NotReady)

Original deadline: 259,201
Effective deadline after change: 1,209,601
Additional wait: 950,400 seconds (11 days)

Also confirmed: setting `unlockPeriod = 0` allows **instant finalization** of all pending unlocks (reverse direction).

Affected Code
contracts/CapitalPool.sol - setUnlockPeriod()(L849), finalizeUnlock()(L415)

Impacts

Same as DO-02: admin adjustments to unlockPeriod retroactively extend or shorten all pending unlock deadlines for capital pool users.

Remediation
Same optional enhancement as DO-02: snapshot the deadline at lock time.

Retest
Acknowledged as intended behavior under the trusted-admin governance model.

Bug ID #DO-04 [Acknowledged]

Unrestricted receive() Absorbs Native PLS into Holder Rewards

Vulnerability Type
Health Check

Severity
Design Observation

Description
The VouchStaking contract accepts native PLS from any sender via an unrestricted receive() function. On the next state-changing call, _calculateHolderRewardDeltas wraps all native PLS to WPLS and treats the surplus above the tracked baseline as distributable holder rewards. Any PLS sent to the contract is absorbed and redistributed to VOUCH stakers. The fuzzer confirmed that a single PLS transfer (~0.0065 ether) is accepted and held in address(staking).balance, then distributed as holder rewards on the next state-changing call. No attacker profits: the sender donates their own funds, which are redistributed to stakers. However, accidentally sent PLS cannot be recovered.

Affected Code
contracts/VouchStaking.sol - receive()(L1834), _calculateHolderRewardDeltas()(L1570)

Impacts
No attacker profit exists. The sender donates their own funds. Accidentally sent PLS cannot be recovered and is permanently absorbed into the holder reward stream. The open receive() ensures compatibility with WPLS unwrapping flows and native PLS from the vouch distributor.

Remediation
If restricting to known senders is preferred: add a sender check in receive() to only accept PLS from the WPLS token address (e.g., if (msg.sender != address(wplsToken)) revert PlsTransferFailed()).

Retest
Acknowledged. The contract natively receives PLS from the vouch distributor, and it must be assumed to be holder rewards. If a user sends PLS there, it will be added to the rewards and absorbed. There is no practical way around this.

Bug ID #DO-05 [Acknowledged]

exit() Permanently Locks Forfeited Holder Rewards in Vault

Vulnerability Type
Health Check

Severity
Design Observation

Description
When a user calls exit(), pending holder rewards are forfeited by snapshotting accumulator values to current globals without claiming. The forfeited rewards remain permanently locked in HolderRewardsVault. Global accumulators are not adjusted downward, so the proportional allocation to the exited user is never redistributable. HolderRewardsVault.recoverToken() (L61) explicitly blocks recovery of VOUCH/VPLS/WPLS. This is documented behavior: exit() is designed as an emergency mechanism that prioritizes immediate principal recovery over reward optimization. The permanent value lockup is an accepted trade-off.

Affected Code
contracts/VouchStaking.sol - exit() (L391-433), contracts/HolderRewardsVault.sol - recoverToken() (L61)

Impacts
Forfeited holder rewards are permanently locked in the vault and cannot be recovered or redistributed. This only applies to the emergency exit() function, which is intended for catastrophic scenarios where the reward pool may be broken.

Remediation
If reclaiming orphaned value is desired, either (a) claim pending holder rewards before forfeiting in exit(), or (b) add an admin function to periodically sweep orphaned rewards from the vault.

Retest
Acknowledged. The exit() function exists for catastrophic scenarios where the reward pool is broken and users need to retrieve their original principal. The rewards would likely be stuck regardless in such scenarios.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com